

coursework_01

February 3, 2023

1 Coursework 1: Image filtering

In this coursework you will practice techniques for image filtering. The coursework includes coding questions and written questions. Please read both the text and the code in this notebook to get an idea what you are expected to implement.

1.1 What to do?

- Complete and run the code using `jupyter-lab` or `jupyter-notebook` to get the results.
- Export (File | Save and Export Notebook As...) the notebook as a PDF file, which contains your code, results and answers, and upload the PDF file onto [Scientia](#).
- Instead of clicking the Export button, you can also run the following command instead:
`jupyter nbconvert coursework_01_solution.ipynb --to pdf`
- If Jupyter complains about some problems in exporting, it is likely that pandoc (<https://pandoc.org/installing.html>) or latex is not installed, or their paths have not been included. You can install the relevant libraries and retry. Alternatively, use the Print function of your browser to export the PDF file.
- If Jupyter-lab does not work for you at the end (we hope not), you can use Google Colab to write the code and export the PDF file.

1.2 Dependencies:

You need to install Jupyter-Lab (https://jupyterlab.readthedocs.io/en/stable/getting_started/installation.html) and other libraries used in this coursework, such as by running the command: `pip3 install [package_name]`

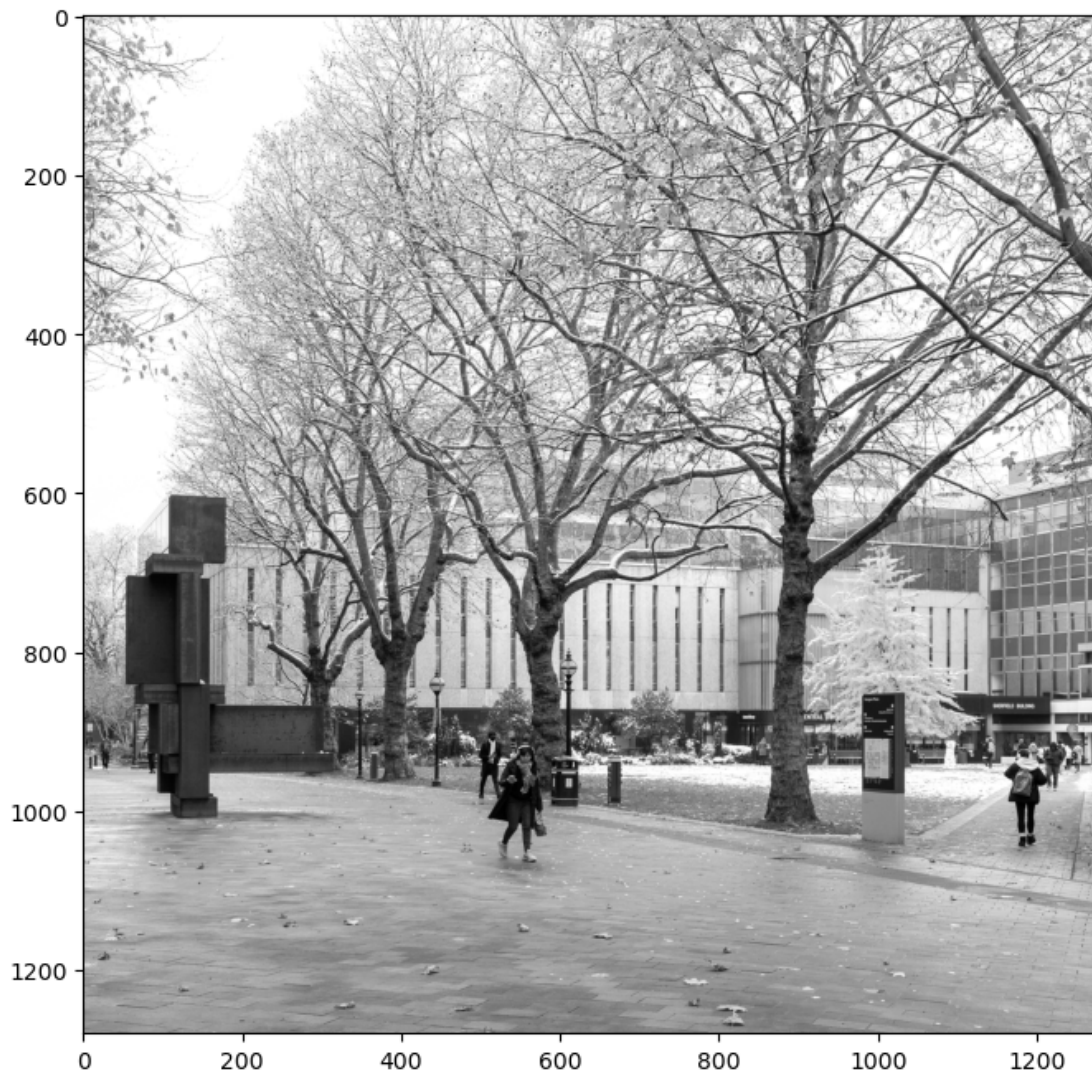
```
[ ]: # Import libraries (provided)
import imageio.v3 as imageio
import numpy as np
import matplotlib.pyplot as plt
import noise
import scipy
import scipy.signal
import math
import time
```

1.3 1. Moving average filter (20 points).

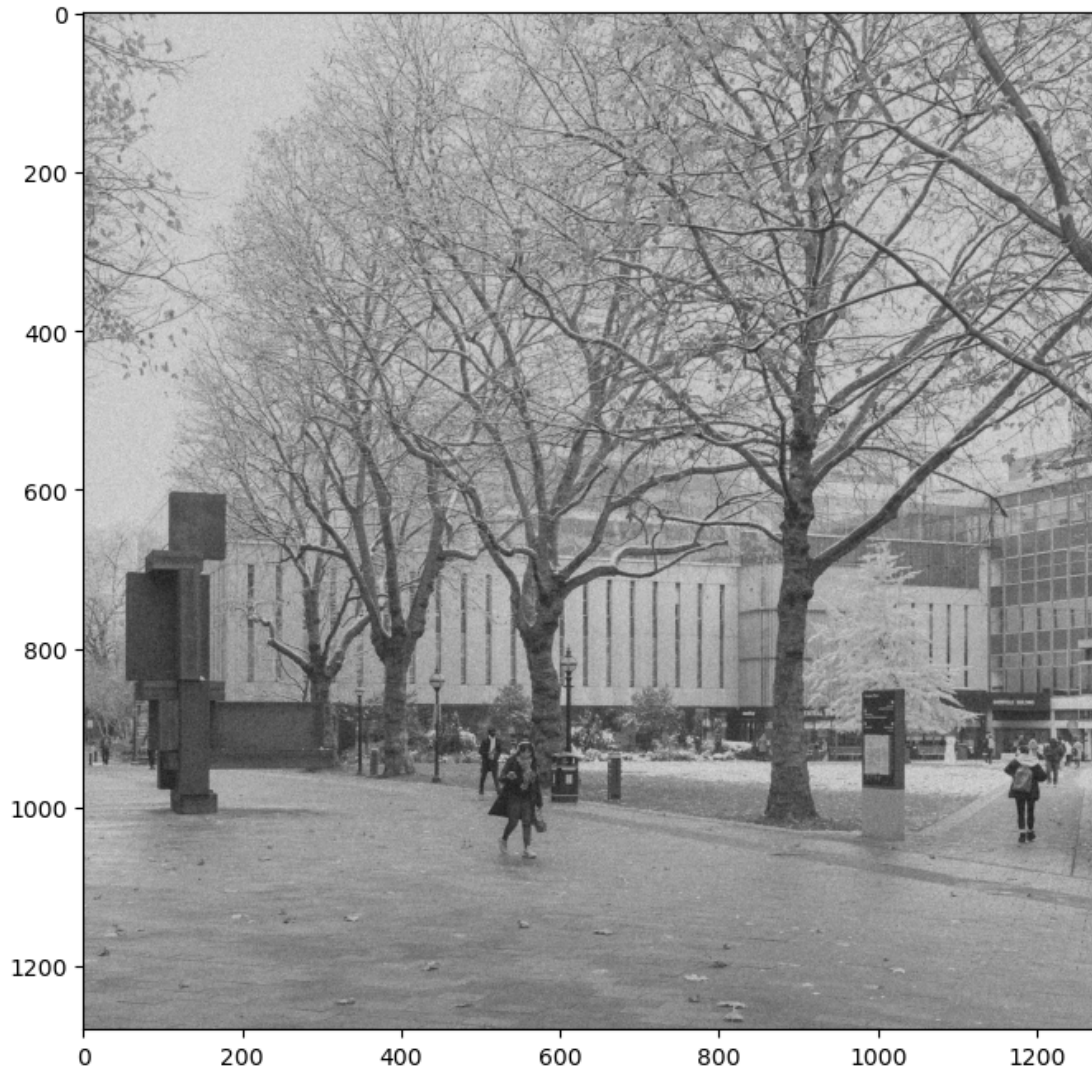
Read the provided input image, add noise to the image and design a moving average filter for denoising.

You are expected to design the kernel of the filter and then perform 2D image filtering using the function `scipy.signal.convolve2d()`.

```
[ ]: # Read the image (provided)
image = imageio.imread('campus_snow.jpg')
plt.imshow(image, cmap='gray')
plt.gcf().set_size_inches(8, 8)
```



```
[ ]: # Corrupt the image with Gaussian noise (provided)
image_noisy = noise.add_noise(image, 'gaussian')
plt.imshow(image_noisy, cmap='gray')
plt.gcf().set_size_inches(8, 8)
```



1.3.1 Note: from now on, please use the noisy image as the input for the filters.

1.3.2 1.1 Filter the noisy image with a 3x3 moving average filter. Show the filtering results.

```
[ ]: # Design the filter h
#### Insert your code ####
h = np.full((3, 3), 1 / 9)
```

```

# Convolve the corrupted image with h using scipy.signal.convolve2d function
### Insert your code ###
image_filtered = scipy.signal.convolve2d(image_noisy, h, mode="same")

# Print the filter (provided)
print('Filter h:')
print(h)

# Display the filtering result (provided)
plt.imshow(image_filtered, cmap='gray')
plt.gcf().set_size_inches(8, 8)

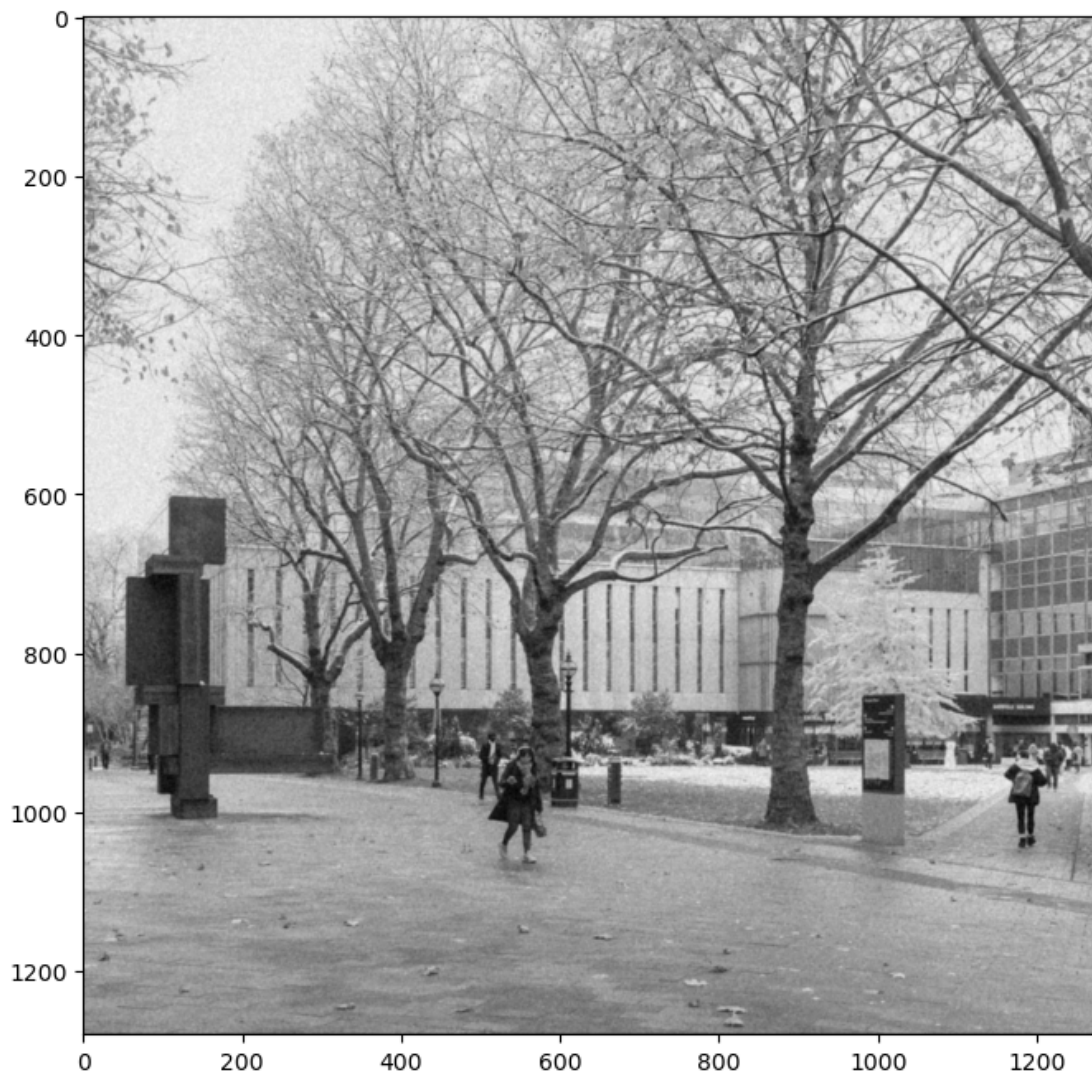
```

Filter h:

```

[[0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]]

```



1.3.3 1.2 Filter the noisy image with a 11x11 moving average filter.

```
[ ]: # Design the filter h
    ### Insert your code ###
    h = np.full((11, 11), 1 / 11)

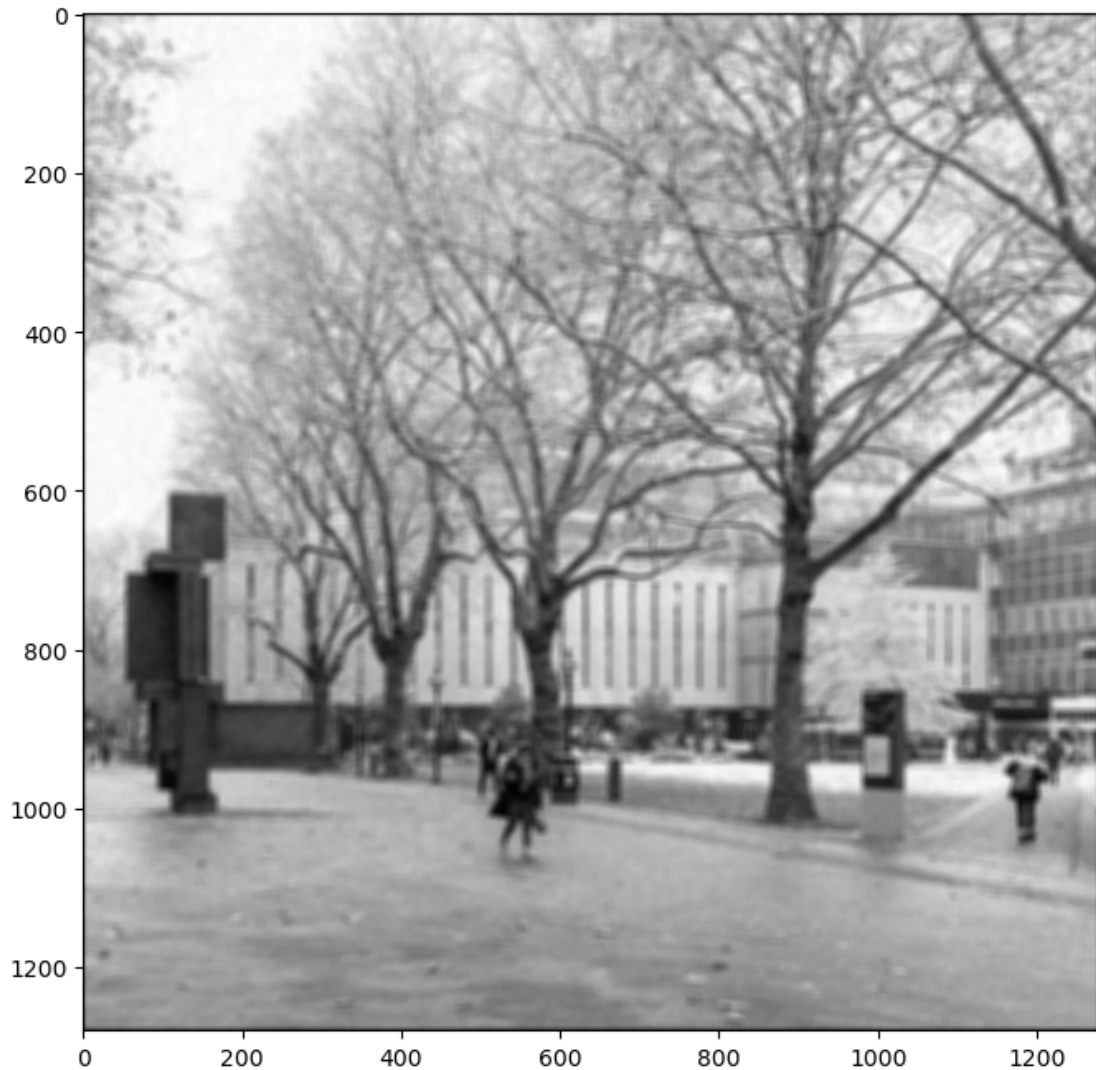
    # Convolve the corrupted image with h using scipy.signal.convolve2d function
    ### Insert your code ###
    image_filtered = scipy.signal.convolve2d(image_noisy, h, mode="same")

    # Print the filter (provided)
    print('Filter h:')
    print(h)

    # Display the filtering result (provided)
    plt.imshow(image_filtered, cmap='gray')
    plt.gcf().set_size_inches(8, 8)
```

Filter h:

```
[[0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]
 [0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
  0.09090909 0.09090909 0.09090909 0.09090909 0.09090909]]
```



1.3.4 1.3 Comment on the filtering results. How do different kernel sizes influence the filtering results?

Insert your answer

The kernel size determines the degree of smoothing applied to an image. Large kernels result in a stronger smoothing effect (i.e. they remove more noise from the image), but this comes at the cost of losing the original image details (there is more blurring in the filtered image). On the contrary, smaller kernels result in a weaker smoothing effect (i.e. they remove less noise from the image), but the original image details are better preserved (there is less blurring in the filtered image).

1.4 2. Edge detection (56 points).

Perform edge detection using Sobel filtering, as well as Gaussian + Sobel filtering.

1.4.1 2.1 Implement 3x3 Sobel filters and convolve with the noisy image.

```
[ ]: # Design the filters
#### Insert your code ####
sobel_x = np.array([[1, 0, -1],
                    [2, 0, -2],
                    [1, 0, -1]])
sobel_y = np.array([[1, 2, 1],
                    [0, 0, 0],
                    [-1, -2, -1]])

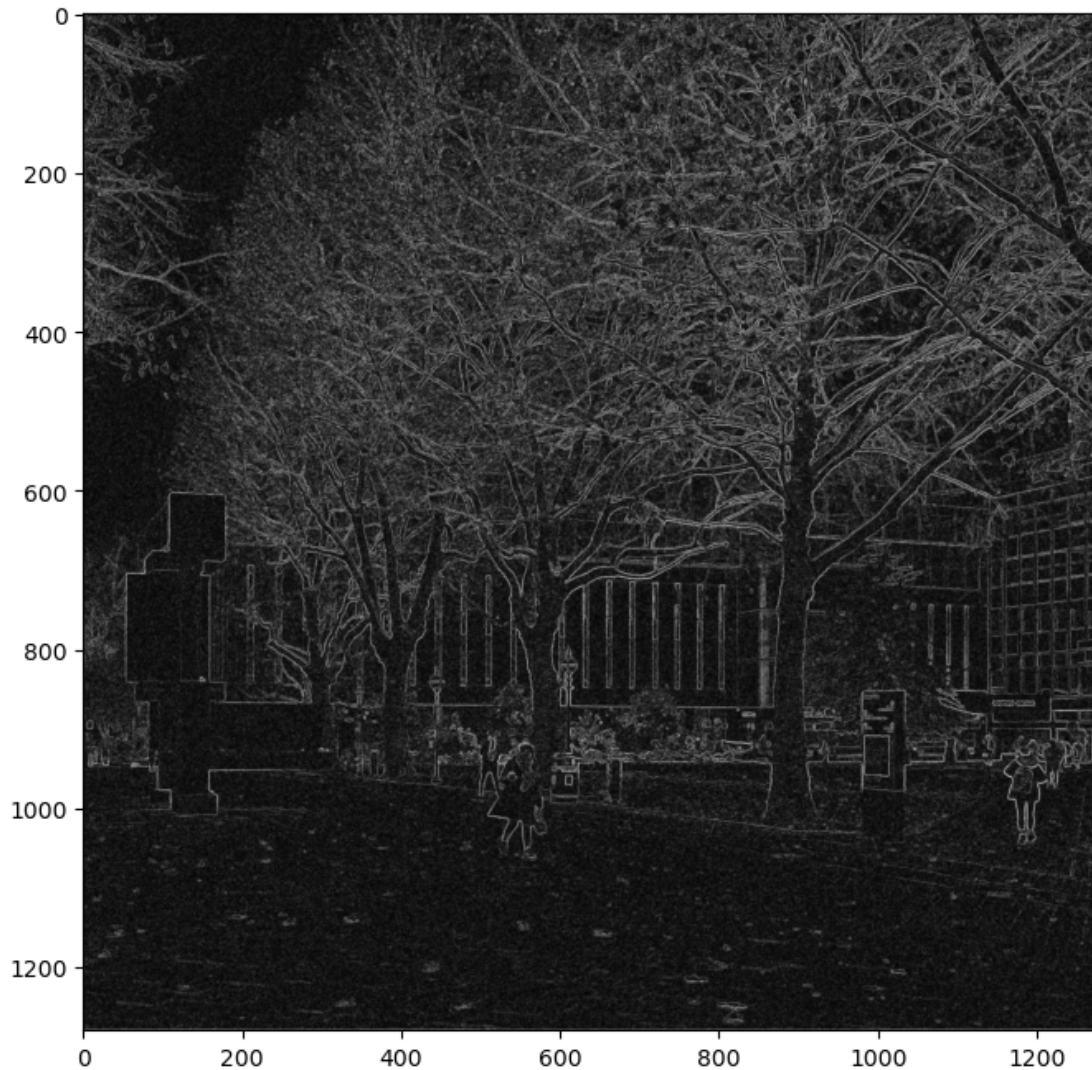
# Image filtering
#### Insert your code ####
filtered_x = scipy.signal.convolve2d(image_noisy, sobel_x, mode="same")
filtered_y = scipy.signal.convolve2d(image_noisy, sobel_y, mode="same")

# Calculate the gradient magnitude
#### Insert your code ####
grad_mag = np.sqrt(filtered_x ** 2 + filtered_y ** 2)

# Print the filters (provided)
print('sobel_x:')
print(sobel_x)
print('sobel_y:')
print(sobel_y)

# Display the magnitude map (provided)
plt.imshow(grad_mag, cmap='gray')
plt.gcf().set_size_inches(8, 8)
```

```
sobel_x:
[[ 1  0 -1]
 [ 2  0 -2]
 [ 1  0 -1]]
sobel_y:
[[ 1  2  1]
 [ 0  0  0]
 [-1 -2 -1]]
```



1.4.2 2.2 Implement a function that generates a 2D Gaussian filter given the parameter σ .

```
[ ]: # Design the Gaussian filter
def gaussian_filter_2d(sigma, k=3):
    # sigma: the parameter sigma in the Gaussian kernel (unit: pixel)
    #
    # return: a 2D array for the Gaussian kernel

    ### Insert your code ###

    def elem_at_index(i, j):
        i_new = i - k * sigma
```

```

        j_new = j - k * sigma
        return math.exp(- (i_new ** 2 + j_new ** 2) / (2 * (sigma ** 2))) / (2 *
↳ math.pi * (sigma ** 2))

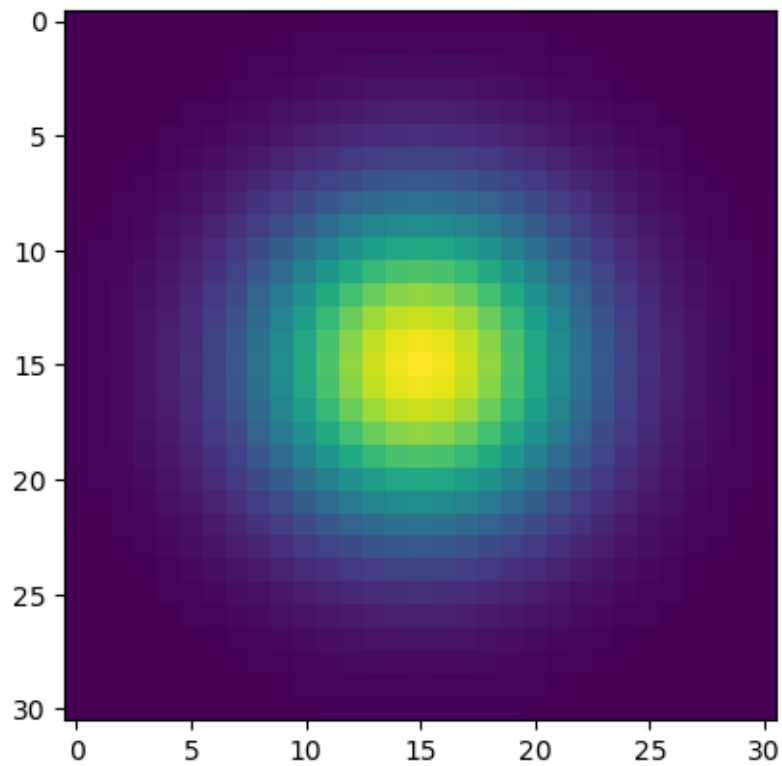
    h = np.fromfunction(np.vectorize(elem_at_index), (2 * k * sigma + 1, 2 * k
↳ sigma + 1))

    return h

# Visualise the Gaussian filter when sigma = 5 pixel (provided)
sigma = 5
h = gaussian_filter_2d(sigma)
plt.imshow(h)

```

```
[ ]: <matplotlib.image.AxesImage at 0x1fdf6d5ed30>
```



1.4.3 2.3 Perform Gaussian smoothing ($\sigma = 5$ pixels) and evaluate the computational time for Gaussian smoothing. After that, perform Sobel filtering and show the gradient magnitude map.

```
[ ]: # Construct the Gaussian filter
    ### Insert your code ###
    sigma = 5
    h = gaussian_filter_2d(sigma)

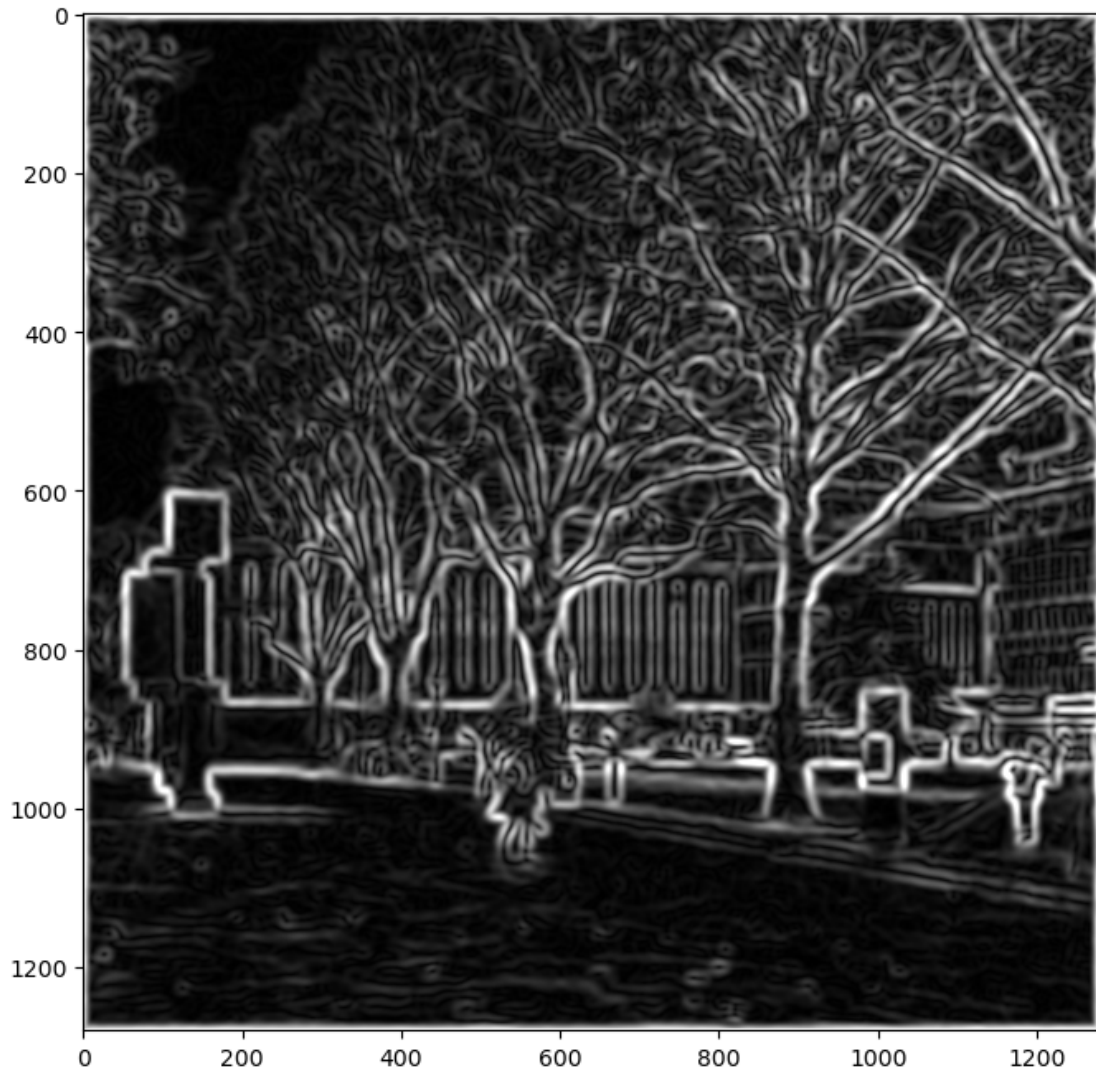
    # Perform Gaussian smoothing and count time
    ### Insert your code ###
    start = time.time()
    image_smoothed = scipy.signal.convolve2d(image_noisy, h, mode="same")
    duration = time.time() - start
    print('Gaussian smoothing takes {} seconds.'.format(duration))

    # Image filtering
    ### Insert your code ###
    filtered_x = scipy.signal.convolve2d(image_smoothed, sobel_x, mode="same")
    filtered_y = scipy.signal.convolve2d(image_smoothed, sobel_y, mode="same")

    # Calculate the gradient magnitude
    ### Insert your code ###
    grad_mag = np.sqrt(filtered_x ** 2 + filtered_y ** 2)

    # Display the gradient magnitude map (provided)
    plt.imshow(grad_mag, cmap='gray', vmin=0, vmax=100)
    plt.gcf().set_size_inches(8, 8)
```

Gaussian smoothing takes 3.4285972118377686 seconds.



1.4.4 2.4 Implement a function that generates a 1D Gaussian filter given the parameter σ . Generate 1D Gaussian filters along x-axis and y-axis respectively.

```
[ ]: # Design the Gaussian filter
def gaussian_filter_1d(sigma, k = 3):
    # sigma: the parameter sigma in the Gaussian kernel (unit: pixel)
    #
    # return: a 1D array for the Gaussian kernel

    ### Insert your code ###

    def elem_at_index(i):
```

```

        i_new = i - k * sigma
        return math.exp(- i_new ** 2 / (2 * (sigma ** 2))) / (math.sqrt(2 *
↪math.pi) * sigma)

    h = np.fromfunction(np.vectorize(elem_at_index), (2 * k * sigma + 1,))

    return h

# sigma = 5 pixel (provided)
sigma = 5
filter_1d = gaussian_filter_1d(sigma)

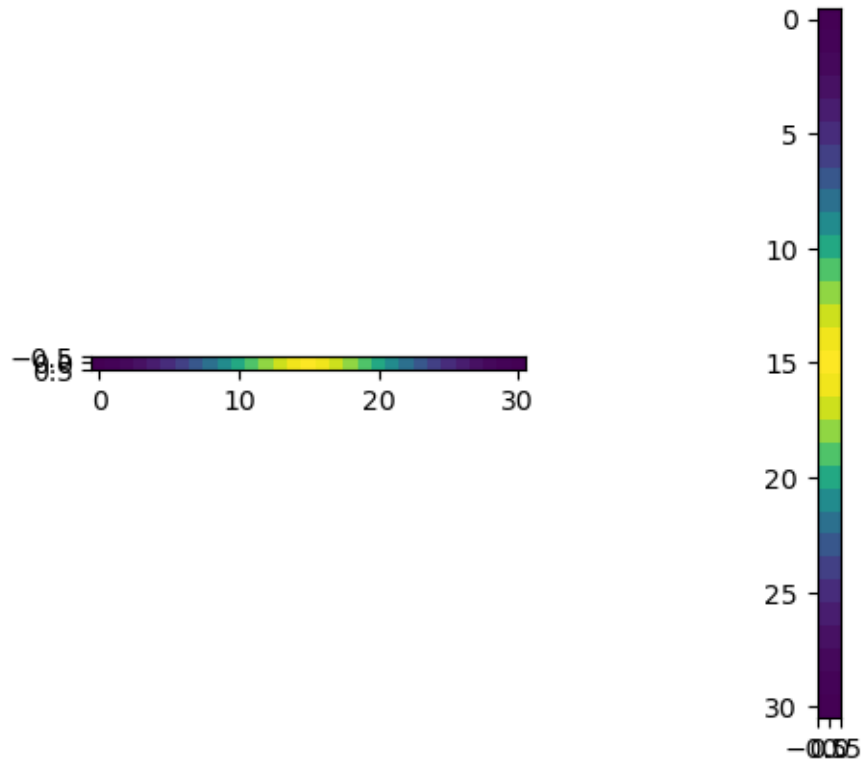
# The Gaussian filter along x-axis. Its shape is (1, sz).
### Insert your code ###
h_x = np.expand_dims(filter_1d, axis=0)

# The Gaussian filter along y-axis. Its shape is (sz, 1).
### Insert your code ###
h_y = np.expand_dims(filter_1d, axis=1)

# Visualise the filters (provided)
plt.subplot(1, 2, 1)
plt.imshow(h_x)
plt.subplot(1, 2, 2)
plt.imshow(h_y)

```

```
[ ]: <matplotlib.image.AxesImage at 0x1fdf6df7f70>
```



1.4.5 2.6 Perform Gaussian smoothing ($\sigma = 5$ pixels) using two separable filters and evaluate the computational time for separable Gaussian filtering. After that, perform Sobel filtering, show the gradient magnitude map and check whether it is the same as the previous one without separable filtering.

```
[ ]: # Perform separable Gaussian smoothing and count time
    ### Insert your code ###
    start = time.time()
    image_smoothed = scipy.signal.convolve2d(image_noisy, h_x, mode="same")
    image_smoothed = scipy.signal.convolve2d(image_smoothed, h_y, mode="same")
    duration = time.time() - start
    print('Gaussian smoothing using separate filtering takes {} seconds.'.
          format(duration))

    # Image filtering
    ### Insert your code ###
    filtered_x = scipy.signal.convolve2d(image_smoothed, sobel_x, mode="same")
    filtered_y = scipy.signal.convolve2d(image_smoothed, sobel_y, mode="same")

    # Calculate the gradient magnitude
    ### Insert your code ###
    grad_mag2 = np.sqrt(filtered_x ** 2 + filtered_y ** 2)
```

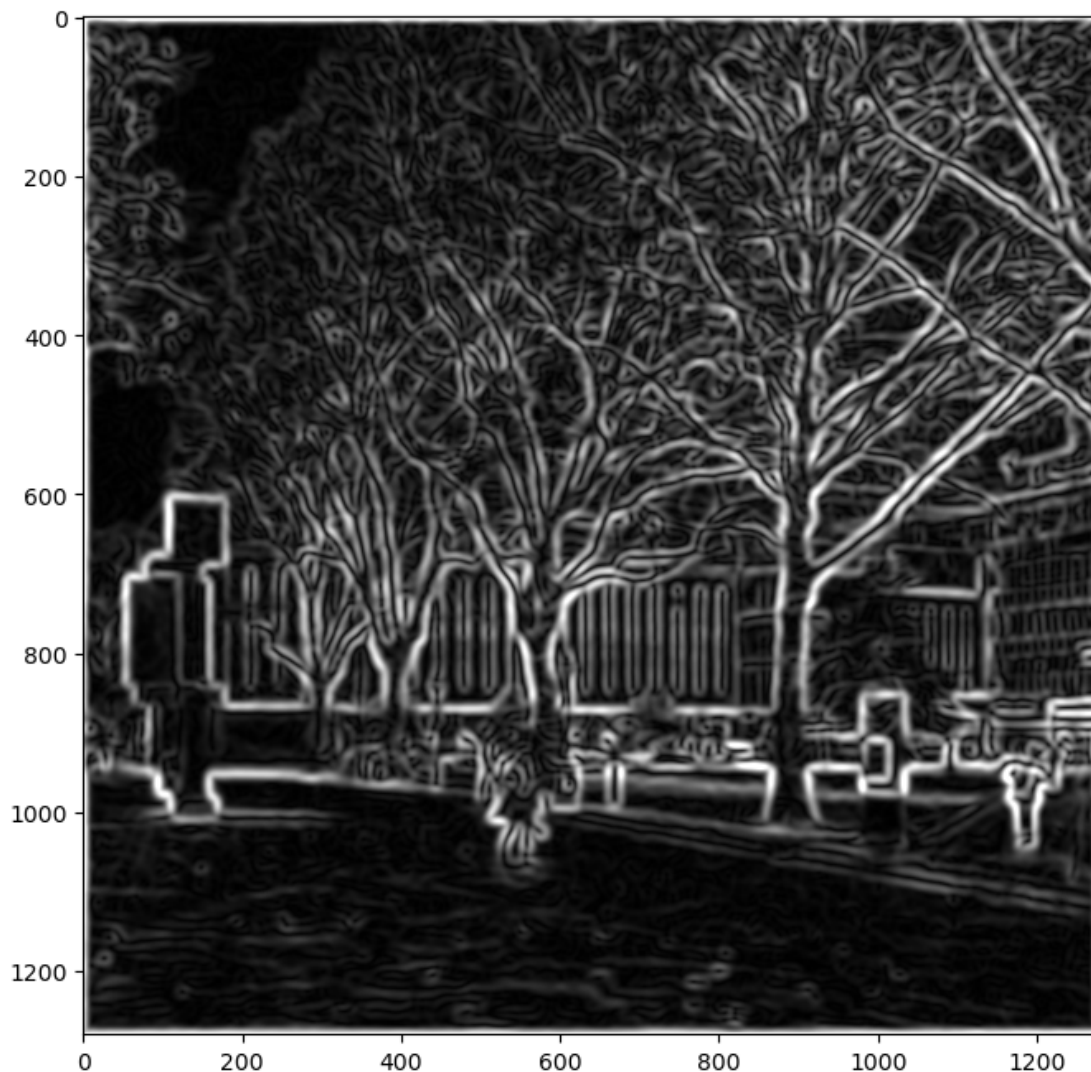
```

# Display the gradient magnitude map (provided)
plt.imshow(grad_mag2, cmap='gray', vmin=0, vmax=100)
plt.gcf().set_size_inches(8, 8)

# Check the difference between the current gradient magnitude map
# and the previous one produced without separable filtering. You
# can report the mean difference between the two.
### Insert your code ###
mean_difference = np.mean(grad_mag2 - grad_mag)
print('The mean difference between the gradient magnitude maps is: {}'.format(mean_difference))

```

Gaussian smoothing using separate filtering takes 0.4310023784637451 seconds.
The mean difference between the gradient magnitude maps is:
6.245468666925173e-15



1.4.6 2.7 Comment on the Gaussian + Sobel filtering results and the computational time.

Insert your answer

Gaussian filters are used for reducing noise in an image, while Sobel filters are used for edge detection. This is why they work together so well: The Gaussian filter is applied first to remove irrelevant noise, which makes it easier for the Sobel filter to detect edges.

For Gaussian smoothing, we get the following computational times (using $k = 3$):

- one filter: \$ 3.4 \$ seconds
- separable filters: \$ 0.4 \$ seconds

When using separate filters, the Gaussian smoothing can be split into two separate filters (one for the horizontal direction and one for the vertical direction). This reduces the complexity from $O(N^2K^2)$ to $O(N^2K)$, which is a significant improvement in performance.

1.5 3. Challenge: Implement 2D image filters using Pytorch (24 points).

[Pytorch](#) is a machine learning framework that supports filtering and convolution.

The [Conv2D](#) operator takes an input array of dimension $N \times C_1 \times X \times Y$, applies the filter and outputs an array of dimension $N \times C_2 \times X \times Y$. Here, since we only have one image with one colour channel, we will set $N=1$, $C_1=1$ and $C_2=1$. You can read the documentation of [Conv2D](#) for more detail.

```
[ ]: # Import libraries (provided)
import torch
```

1.5.1 3.1 Expand the dimension of the noisy image into $1 \times 1 \times X \times Y$ and convert it to a Pytorch tensor.

```
[ ]: # Expand the dimension of the numpy array
### Insert your code ###
expanded_noisy = image_noisy[np.newaxis, np.newaxis, :, :]

# Convert to a Pytorch tensor using torch.from_numpy
### Insert your code ###
expanded_noisy = torch.from_numpy(expanded_noisy)
```

1.5.2 3.2 Create a Pytorch Conv2D filter, set its kernel to be a 2D Gaussian filter and perform filtering.

```
[ ]: # A 2D Gaussian filter when sigma = 5 pixel (provided)
sigma = 5
h = gaussian_filter_2d(sigma)

# Create the Conv2D filter
```

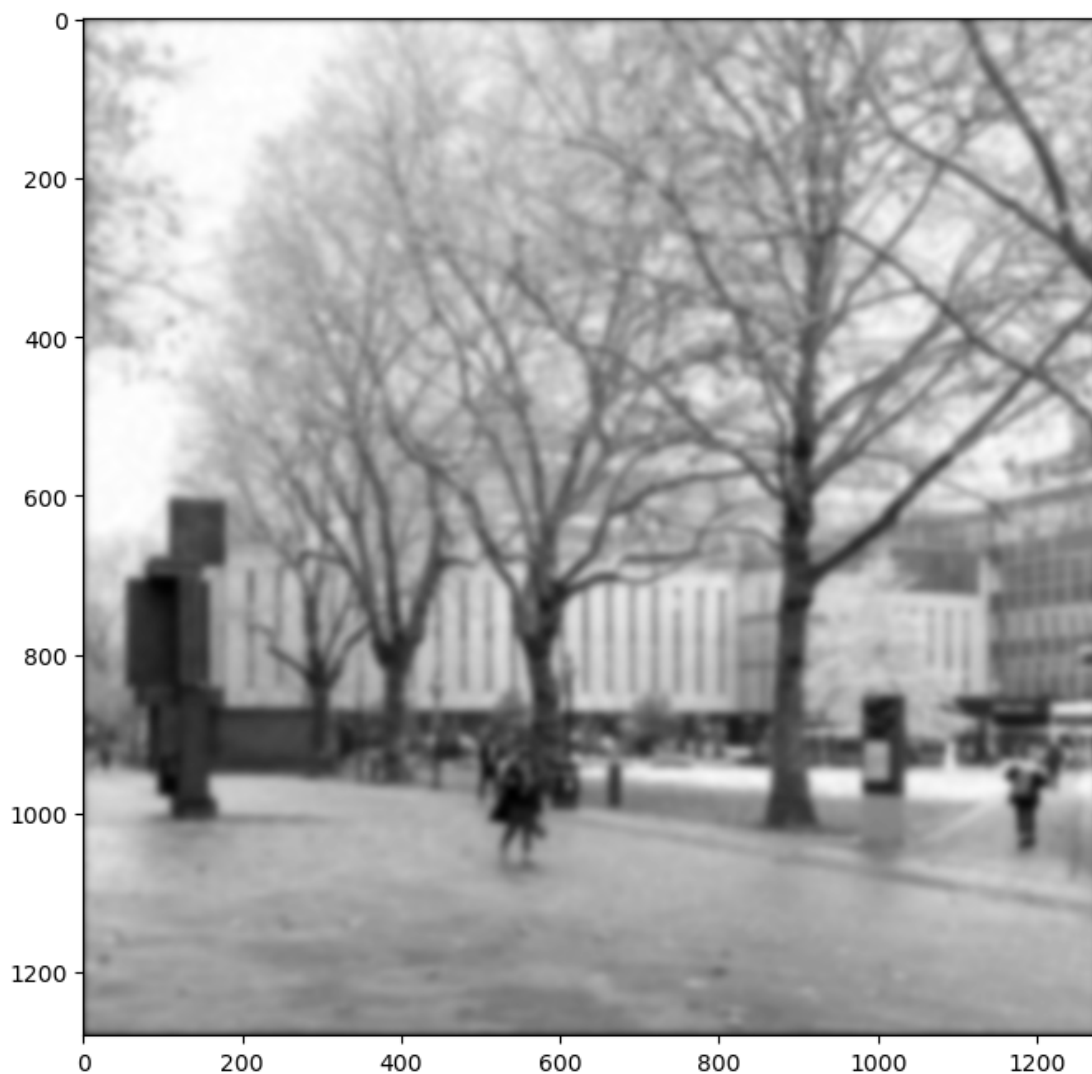
```

### Insert your code ###
filter = torch.from_numpy(h[np.newaxis, np.newaxis, :, :])

# Filtering
### Insert your code ###
image_filtered = torch.nn.functional.conv2d(expanded_noisy, filter,
padding="same")
image_filtered = image_filtered.squeeze()

# Display the filtering result (provided)
plt.imshow(image_filtered, cmap='gray')
plt.gcf().set_size_inches(8, 8)

```



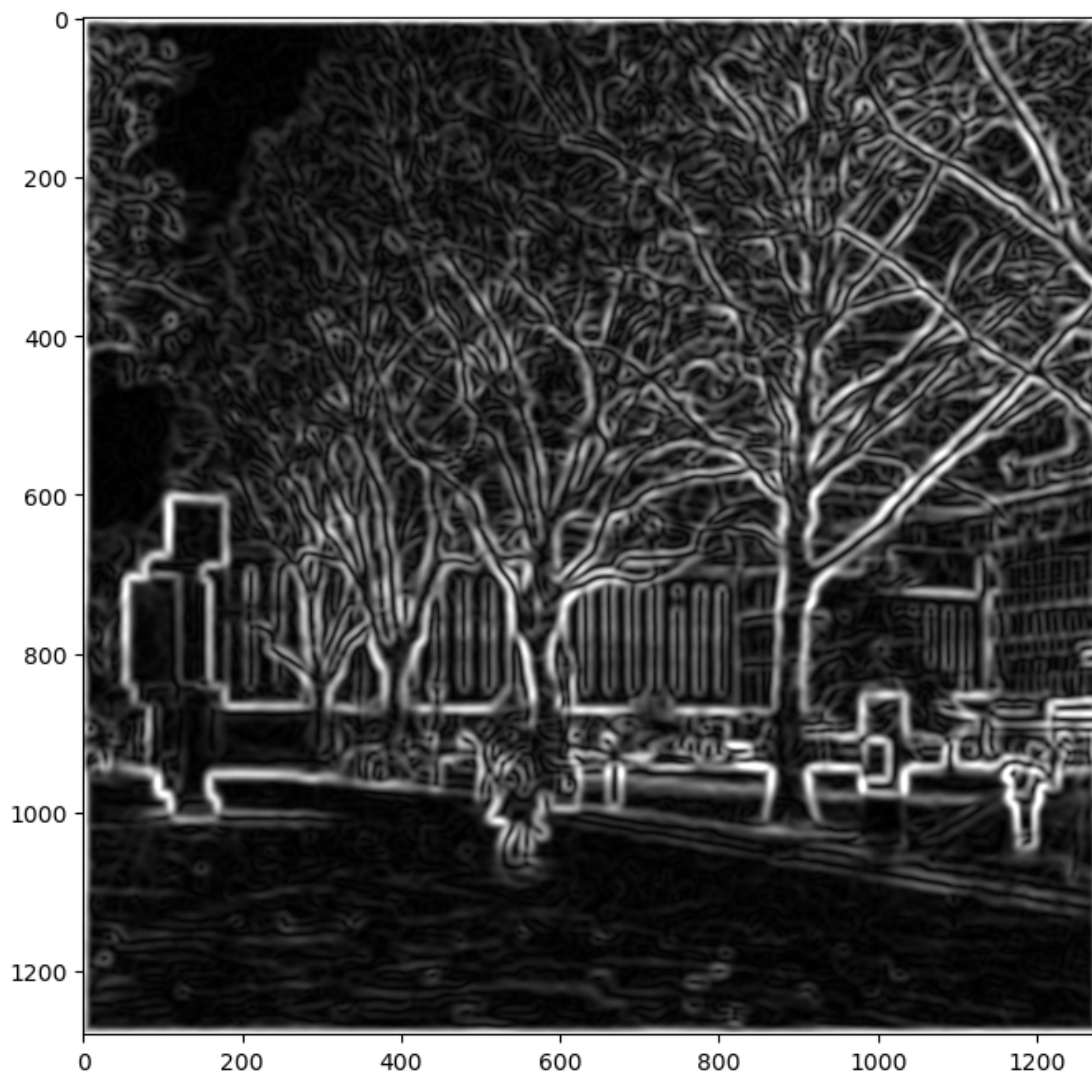
1.5.3 3.3 Implement Pytorch Conv2D filters to perform Sobel filtering on Gaussian smoothed images, show the gradient magnitude map.

```
[ ]: # Create Conv2D filters
#### Insert your code ####
torch_sobel_x = torch.from_numpy(sobel_x[np.newaxis, np.newaxis, :, :]).double()
torch_sobel_y = torch.from_numpy(sobel_y[np.newaxis, np.newaxis, :, :]).double()

# Perform filtering
#### Insert your code ####
expanded_image_filtered = image_filtered[np.newaxis, np.newaxis, :, :]
filtered_x = torch.nn.functional.conv2d(expanded_image_filtered, torch_sobel_x,
padding="same")
filtered_x = filtered_x.squeeze()
filtered_y = torch.nn.functional.conv2d(expanded_image_filtered, torch_sobel_y,
padding="same")
filtered_y = filtered_y.squeeze()

# Calculate the gradient magnitude map
#### Insert your code ####
grad_mag3 = np.sqrt(filtered_x ** 2 + filtered_y ** 2)

# Visualise the gradient magnitude map (provided)
plt.imshow(grad_mag3, cmap='gray', vmin=0, vmax=100)
plt.gcf().set_size_inches(8, 8)
```



[]: